

ALGORITMA DAN STRUKTUR DATA(ASD)

TUGAS PRAKTIKUM 9 GRAPH



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C

PROGRAM STUDI TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – GRAP

Berdasarkan gambar graph pada lampiran, implementasikan graph dan algoritma rute terpendek menggunakan algoritma Warshall dan Dijkstra. Program dapat menampilkan urutan rute menggunakan stack dan beban minimal. Tuliskan source code, hasil running program, dan analisis. Berikan kesimpulan hasil praktikum.

Jawaban:

Kode:

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. #define max 6
5. #define inf 99999
6.
7. typedef struct {
8.     int data[100];
9.     int top;
10. } stack;
11.
12. void initstack(stack *s);
13. void push(stack *s, int value);
14. int pop(stack *s);
15. int getmindist(int dist[], int visited[]);
16. void dijkstra(int graph[max][max], int src, int dest);
17. void warshall(int graph[max][max], int src, int dest);
18.
19. int main() {
20.     int graph[max][max] = {
21.         {0, 4, 3, inf, inf, inf},
22.         {inf, 0, 5, 2, inf, inf},
23.         {inf, inf, 0, 7, inf, inf},
24.         {inf, inf, inf, 0, 2, inf},
25.         {4, 4, inf, inf, 0, 6},
26.         {inf, inf, inf, inf, inf, 0}
27.     };
28.
29.     int start = 0;
30.     int end = 5;
31.
32.     printf("praktikum graph & shortest path\n");
33.     printf("node asal (input) : %d\n", start);
34.     printf("node tujuan (input): %d\n", end);
35.
```

```

36.     dijkstra(graph, start, end);
37.     warshall(graph, start, end);
38.
39.     return 0;
40. }
41.
42. void initstack(stack *s) {
43.     s->top = -1;
44. }
45.
46. void push(stack *s, int value) {
47.     s->data[++(s->top)] = value;
48. }
49.
50. int pop(stack *s) {
51.     if (s->top == -1) return -1;
52.     return s->data[(s->top)--];
53. }
54.
55. int getmindist(int dist[], int visited[]) {
56.     int min = inf, min_idx = -1;
57.     for (int v = 0; v < max; v++) {
58.         if (!visited[v] && dist[v] <= min) {
59.             min = dist[v];
60.             min_idx = v;
61.         }
62.     }
63.     return min_idx;
64. }
65.
66. void dijkstra(int graph[max][max], int src, int dest) {
67.     int dist[max], visited[max], parent[max];
68.
69.     for (int i = 0; i < max; i++) {
70.         dist[i] = inf;
71.         visited[i] = 0;
72.         parent[i] = -1;
73.     }
74.
75.     dist[src] = 0;
76.
77.     for (int count = 0; count < max - 1; count++) {
78.         int u = getmindist(dist, visited);

```

```

79.         if (u == -1 || u == inf) break;
80.
81.         visited[u] = 1;
82.
83.         for (int v = 0; v < max; v++) {
84.             if (!visited[v] && graph[u][v] != inf && dist[u] != inf
&& dist[u] + graph[u][v] < dist[v]) {
85.                 dist[v] = dist[u] + graph[u][v];
86.                 parent[v] = u;
87.             }
88.         }
89.     }
90.
91.     printf("\nalgoritma dijkstra\n");
92.     if (dist[dest] == inf) {
93.         printf("tidak ada rute dari %d ke %d\n", src, dest);
94.         return;
95.     }
96.
97.     printf("beban minimal : %d\n", dist[dest]);
98.
99.     stack s;
100.    initstack(&s);
101.    int curr = dest;
102.    while (curr != -1) {
103.        push(&s, curr);
104.        curr = parent[curr];
105.    }
106.
107.    printf("urutan rute   : ");
108.    while (s.top != -1) {
109.        printf("%d", pop(&s));
110.        if (s.top != -1) printf(" -> ");
111.    }
112.    printf("\n");
113. }
114.
115. void warshall(int graph[max][max], int src, int dest) {
116.     int dist[max][max];
117.     int next[max][max];
118.
119.     for (int i = 0; i < max; i++) {
120.         for (int j = 0; j < max; j++) {

```

```

121.         dist[i][j] = graph[i][j];
122.         if (graph[i][j] != inf && i != j) {
123.             next[i][j] = j;
124.         } else {
125.             next[i][j] = -1;
126.         }
127.     }
128. }
129.
130. for (int k = 0; k < max; k++) {
131.     for (int i = 0; i < max; i++) {
132.         for (int j = 0; j < max; j++) {
133.             if (dist[i][k] != inf && dist[k][j] != inf &&
dist[i][k] + dist[k][j] < dist[i][j]) {
134.                 dist[i][j] = dist[i][k] + dist[k][j];
135.                 next[i][j] = next[i][k];
136.             }
137.         }
138.     }
139. }
140.
141. printf("\nalgoritma floyd-warshall\n");
142. if (dist[src][dest] == inf) {
143.     printf("tidak ada rute dari %d ke %d\n", src, dest);
144.     return;
145. }
146.
147. printf("beban minimal : %d\n", dist[src][dest]);
148.
149. int temppath[max];
150. int count = 0;
151. int curr = src;
152.
153. while (curr != dest) {
154.     temppath[count++] = curr;
155.     curr = next[curr][dest];
156.     if (curr == -1) break;
157. }
158. temppath[count++] = dest;
159.
160. stack s;
161. initstack(&s);
162. for (int i = count - 1; i >= 0; i--) {

```

```

163.         push(&s, temppath[i]);
164.     }
165.
166.     printf("urutan rute   : ");
167.     while (s.top != -1) {
168.         printf("%d", pop(&s));
169.         if (s.top != -1) printf(" -> ");
170.     }
171.     printf("\n");
172. }

```

Hasil Kode:

```

praktikum graph & shortest path
node asal (input)  : 0
node tujuan (input): 5

    algoritma dijkstra
    beban minimal  : 14
    urutan rute    : 0 -> 1 -> 3 -> 4 -> 5

    algoritma floyd-warshall
    beban minimal  : 14
    urutan rute    : 0 -> 1 -> 3 -> 4 -> 5

```

Analisis Kode:

Program ini berjalan dengan pertama-tama menyiapkan data peta graf menggunakan matriks dua dimensi di dalam fungsi utama, di mana angka-angka tersebut mewakili jarak atau beban antar titik. jika tidak ada jalan langsung yang menghubungkan dua lokasi, nilainya diisi dengan batas tak terhingga. setelah titik asal dan tujuan ditentukan, program mulai memanggil algoritma dijkstra. alur kerja dijkstra di sini cukup terarah; ia mulai dari titik awal, lalu secara bertahap mencari tetangga terdekat yang belum dikunjungi. setiap kali menemukan jalan alternatif yang total jaraknya lebih pendek, program akan mencatat jarak baru tersebut dan menyimpan informasi titik asalnya ke dalam array parent. proses pencarian ini terus berulang sampai seluruh jalur terevaluasi, sehingga rute terpendek nantinya bisa dilacak mundur.

Setelah dijkstra mencetak hasil, program berlanjut menjalankan algoritma floyd-warshall. alur fungsi ini berbeda karena ia bekerja secara menyeluruh dengan langsung mengevaluasi semua kemungkinan rute di seluruh graf menggunakan sistem perulangan. inti dari alur kerjanya adalah memeriksa apakah perjalanan dari titik asal ke titik tujuan akan menjadi lebih cepat jika melewati sebuah titik perantara atau transit. jika iya, jarak terpendeknya diperbarui dan rutenya dicatat. pada bagian akhir dari masing-masing fungsi algoritma tersebut, program memanfaatkan struktur data stack untuk merapikan rute akhir. karena rute yang didapatkan dari proses pelacakan bentuknya berjalan mundur dari tujuan, semua titik dimasukkan ke dalam stack terlebih dahulu. saat titik-titik tersebut dikeluarkan satu per satu untuk dicetak ke layar,

urutannya otomatis berbalik menjadi rute maju yang benar dari titik awal sampai tujuan.

Kesimpulan:

Berdasarkan percobaan pencarian rute yang telah dilakukan, dapat disimpulkan bahwa algoritma dijkstra maupun floyd-warshall sama-sama berhasil menemukan jalan terpendek dengan sangat akurat. keduanya mampu memberikan hasil komputasi yang identik, baik dari segi angka total beban minimal maupun urutan titik persimpangan yang dilalui. hal ini menjadi bukti nyata bahwa walaupun kedua algoritma tersebut memiliki alur penyelesaian masalah dan cara perhitungan yang berlawanan, keduanya tetap terbukti valid dan bisa diandalkan untuk memecahkan kasus pencarian rute pada graf berbobot.

Dari segi penerapannya, pemilihan algoritma ini hanya tinggal menyesuaikan dengan skala sistem yang dibangun. algoritma dijkstra terbukti jauh lebih efisien, cepat, dan hemat memori jika kita hanya fokus mencari satu rute dari satu titik ke titik lainnya. sementara itu, floyd-warshall sangat cocok jika aplikasi menuntut perhitungan rute secara keseluruhan dari semua titik ke semua titik secara bersamaan. selain pemahaman tentang graf, praktikum ini juga memberikan gambaran jelas mengenai fungsi stack di luar teori dasarnya, yaitu sebagai alat bantu yang sangat praktis untuk membalik urutan data pelacakan mundur agar menjadi informasi yang siap dibaca oleh pengguna.